

Full Marks:50

SET:A

Time: 1Hour 10Minutes

Q1. What will happen?

```
def outer():
    x = 10

    def inner():
        nonlocal y
        y = 20

    inner()
```

outer()

- A)10
- B)20
- C)SyntaxError
- D)NameError

Q2. Why is

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
codes's second handler never
reached?
```

- A)Exception is a superclass of ZeroDivisionError
- B)except blocks execute randomly
- C)ZeroDivisionError occurs before Exception
- D)ZeroDivisionError is not an exception

Q3. What is the output?

```
def fun(x):
    return x * 2
```

```
a = fun
print(a(5))
```

- A)10
- B>Error
- C)func(5)
- D)5

Q4. What will be printed?

```
class InvalidAgeError(Exception):
    pass

def check_age(age):
    if age < 18:
        raise InvalidAgeError("Age is
too low")
    return "Allowed"
```

```
try:
    print(check_age(15))
except InvalidAgeError as e:
    print(type(e).__name__)
    print(e)
```

- A)InvalidAgeError
- Age is too low
- B)Exception
- Age is too low
- C)InvalidAgeError
- 15
- D)Nothing, because custom exceptions cannot be caught

Q5. What is the output?

```
def fun(x):
    x += 10
    return x
```

```
x = 5
y = fun(x)
```

print(x, y)

- A)15 5
- B)5 15
- C)15 15
- D)5 5

Q6. What does the following function return?



```
def fun():  
    print("Hello")
```

```
x = fun()
```

```
print(x)
```

- A)Error
- B)None
- C)Hello
- D)Hello None

Q7. What will be the output?

```
x = 10
```

```
def fun():  
    print(x)  
    x = 20
```

```
fun()
```

- A)20
- B)NameError
- C)UnboundLocalError
- D)10

Q8. What will be printed?

```
try:  
    print(10 / 0)  
except ZeroDivisionError:  
    print("A")  
except Exception:  
    print("B")
```

- A)A
- B)B
- C)A B
- D)Nothing

Q9. What will be printed?

```
def fun(a, b):  
    return a + b
```

```
print(fun(10, b=20))
```

- A)SyntaxError
- B)20
- C)30
- D)TypeError

Q10. What will happen?

```
def fun(n):  
    if n == 0:  
        return 1  
    return n * fun(n)
```

```
print(fun(3))
```

- A)TypeError
- B)6
- C)RecursionError
- D)3

Q11. What will be printed?

```
def fun():  
    print("A")  
    return  
    print("B")
```

```
fun()
```

- A)B
- B)Nothing
- C)A
- D)A B

Q12. What will be printed?

```
def make_counter():  
    count = 0
```

```
    def counter():  
        nonlocal count  
        count += 1  
        return count
```

```
    return counter
```

```
a = make_counter()  
b = make_counter()
```

```
print(a())  
print(a())  
print(b())  
print(a())
```

- A)1
- 2
- 3
- 4
- B)1
- 1
- 1



1
C)0
1
0
2
D)1
2
1
3

Q13. What is the output?

```
try:
    print(10 / 0)
except Exception:
    print("A")
except ZeroDivisionError:
    print("B")
```

A)SyntaxError
B)A
C)B
D)A B

Q14. What is the output?

```
try:
    raise ValueError
except ValueError:
    print("A")
    raise
```

A)A, followed by propagation of the same exception
B)No Output
C)A, and execution ends normally
D)B

Q15. What happens when this function is called?

```
def fun(a=10, b):
    return a + b
```

A)SyntaxError
B)TypeError
C)It executes correctly
D)NameError

Q16. What is the purpose of this statement?

```
if __name__ == "__main__":
```

A)It creates the main function
B)It prevents syntax errors

C)It prevents all imports
D)It identifies code intended to run when the file is executed directly

Q17. Which object keeps track of modules that have already been imported during the current Python process?

A)__module__
B)sys.modules
C)sys.path
D)sys.imports

Q18. Suppose test.py contains:

```
print("A")
```

```
if __name__ == "__main__":
    print("B")
```

If another Python program executes:

```
import test
```

what will normally be printed?

A)B
B)Nothing
C)A
D)A and B

Q19. What will be printed?

```
def fun():
    try:
        return 10
    finally:
        print("Finally")
```

```
print(fun())
```

A)Finally followed by 10
B)10 followed by Finally
C)10 only
D)Finally only

Q20. What happens here?

```
class MyError(Exception):
    pass
```

```
def test():
    raise MyError("Something went wrong")
```



```
try:
    test()
except MyError as e:
    print(type(e).__name__)
```

- A)Something went wrong
- B)TypeError
- C)MyError
- D)Exception

Q21. What will normally happen?

```
import math
import math
```

- A)SyntaxError
- B)ImportError
- C)The module is normally loaded and executed twice
- D)The module is normally loaded once and reused

Q22. What happens?

```
def fun(a, b):
    return a + b
```

```
print(fun(10, a=20))
```

- A)SyntaxError
- B)20
- C)30
- D)TypeError

Q23. What will be printed?

```
x = 10
```

```
f = lambda: x
```

```
x = 20
```

```
print(f())
```

- A)NameError
- B)20
- C)10
- D)None

Q24. What is the output?

```
def fun(x):
    if x == 0:
        return
    return x + fun(x - 1)
```

```
print(fun(3))
```

- A)6
- B)None
- C)3
- D)5

**Q25. if test.py has
print("A")**

```
if __name__ == "__main__":
    print("B")
```

If test.py is executed directly, what will be printed?

- A)A followed by B
- B)Nothing
- C)A
- D)B

Q26. What happens when this code executes?

```
def fun():
    print(variable)
    variable = 50
```

```
variable = 10
```

```
fun()
```

- A)UnboundLocalError
- B)50
- C)10
- D)NameError

Q27. What happens here?

```
def fun(a):
    return a
```

```
print(fun())
```

- A)NameError
- B)TypeError
- C)0
- D)None

Q28. What will be printed?

```
from functools import reduce
```

```
numbers = [1, 2, 3, 4]
```

```
result = reduce(
    lambda a, b: a * b,
```



```

    numbers
)

print(result)
A)24
B)0
C)10
D)9
Q29. What happens when this code runs?

def first():
    second()

def second():
    third()

def third():
    raise RuntimeError("Failure")

try:
    first()
except RuntimeError:
    print("Handled")
A)Handled
B)NameError
C)TypeError
D)RuntimeError cannot propagate through functions
Q30. What will happen?

def fun(a, b=10):
    return a + b

print(fun(b=5, a=20))
A)SyntaxError
B)TypeError
C)25
D)15
Q31. What is the output?

def fun():
    try:
        return 10
    finally:
        return 20

print(fun())

```

```

A)20
B)RuntimeError
C)10
D)None
Q32. What happens here?

from math import sqrt

print(sqrt(36))
A)NameError
B)6
C)6.0
D)36
Q33. Suppose module_a.py contains:

print("Module loaded")

value = 10

And another file contains:

import module_a
import module_a

print(module_a.value)

Assuming both imports occur in the same Python process, what is normally printed?
A)ImportError
B)Module loaded
Module loaded
10
C)10
10
D)Module loaded
10
Q34. What is the output?

f = lambda x: x + 5

print(f(10))
A>Error
B)10
C)5
D)15
Q35. What is the output?

```



```
try:
    x = int("abc")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)A
- B)B C
- C)B
- D)A C

Q36. Suppose sample.py contains:

```
x = 10
```

```
def display():
    print("Hello")
```

What will this print?

```
import sample
```

```
print(sample.x)
sample.display()
```

- A)Only Hello
- B)10
Hello
- C)x
display
- D)Error

Q37. What will be printed?

```
def fun(x=[]):
    x.append(10)
    return x
```

```
a = fun()
b = fun()
```

```
print(a)
print(b)
```

- A)Error
- B)[10]
[10, 10]
- C)[10]
[10]

```
D)[10, 10]
[10]
```

Q38. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        return x + 5
```

```
    return inner
```

```
f = outer()
print(f())
```

- A)15
- B)NameError
- C)10
- D)None

Q39. What is the output?

```
def fun():
    try:
        print("A")
        raise ValueError
    except ValueError:
        print("B")
        return 10
    else:
        print("C")
        return 20
    finally:
        print("D")
```

```
print(fun())
```

- A)A
- B
- D
- 10
- B)A
- C
- D
- 20
- C)A
- B
- 10
- D



D)A
B
C
D
10

Q40. What is the output?

```
numbers = [1, 2, 3, 4, 5]
```

```
result = list(
    map(
        lambda x: x * 2,
        filter(lambda x: x % 2 == 0,
            numbers)
    )
)
```

```
print(result)
```

A)[2, 4]
B)[1, 3, 5]
C)[2, 4, 6, 8, 10]
D)[4, 8]

Q41. What will be the output?

```
import math
```

```
print(math.sqrt(25))
```

A)sqrt(25)
B)5.0
C)5
D)25

Q42. What is the output?

```
def outer():
    x = 10
```

```
    def inner():
        nonlocal x
        x = x + 5
        return x
```

```
    inner()
    return x
```

```
print(outer())
```

A)15
B)UnboundLocalError
C)10

D)5

Q43. Suppose the current directory contains a file named random.py. What potential problem can occur with:

```
import random
```

A)Python automatically renames the local file
B)The local file can shadow the intended standard-library module
C)random cannot be imported
D)Python will always reject it

Q44. What is the output?

```
def fun(a=10, b=20):
    return a - b
```

```
print(fun(30))
```

A)10
B)-10
C)20
D)30

Q45. What will be printed?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        print(x)
```

```
    x = 30
    inner()
```

```
outer()
```

A)NameError
B)20
C)30
D)10

Q46. What will be the output?

```
def fun(x):
    try:
        if x < 0:
            raise ValueError("negative")
    return x
```



```
except ValueError:
    return 100
finally:
    print("cleanup")
```

print(fun(-5))

- A)100
- cleanup
- B)cleanup
- 100
- C)negative
- cleanup
- D)cleanup
- 5

Q47. What will be the output?

```
x = 10
```

```
def outer():
    x = 20
```

```
    def inner():
        global x
        x = 30
```

```
    inner()
    print(x)
```

```
outer()
print(x)
```

- A)20
- 10
- B)30
- 30
- C)30
- 20
- D)20
- 30

Q48. What happens here?

```
try:
    raise ValueError("Wrong value")
except ValueError as e:
    print(e)
```

- A)Wrong Value
- B)ValueError
- C)e
- D)Nothing

Q49. What is the major difference between the following?

```
import math
```

and

```
from math import sqrt
```

- A)There is no difference
- B)Both import only sqrt
- C)First provides the module namespace; second imports sqrt directly
- D)First imports only sqrt

Q50. What will be printed?

```
try:
    x = int("10")
except ValueError:
    print("A")
else:
    print("B")
finally:
    print("C")
```

- A)B C
- B)A
- C)A C
- D)B

